

Módulo 9 – Capítulo 08 – Sección 01

Logs de seguridad: qué registrar — inputs, outputs, herramientas invocadas y decisiones

El diseño del sistema de logging de seguridad es una decisión arquitectónica que debe tomarse antes de ir a producción, no como reacción a un incidente. Esta afirmación tiene una razón técnica concreta: añadir logging completo a un sistema en producción que no fue diseñado para ello requiere frecuentemente cambios en el esquema de la base de datos, en el formato de los eventos, y en los controles de acceso a los datos de logging —cambios que son costosos de implementar, riesgosos de desplegar en un sistema que ya está sirviendo tráfico, y que pueden crear brechas temporales de cobertura durante la transición. El sistema de logging diseñado desde el inicio puede ser parte del hot path de cada request sin sorpresas arquitectónicas posteriores.

La tensión fundamental en el diseño del logging es entre la cobertura forense y la privacidad. Registrar inputs y outputs completos de todas las requests proporciona máxima cobertura para análisis forense pero crea riesgos de privacidad —esos logs contienen todo lo que el usuario escribió, potencialmente incluyendo datos sensibles—, costos de almacenamiento significativos en sistemas de alto volumen, y obligaciones adicionales bajo GDPR y HIPAA para la gestión de retención y borrado de esos datos. La solución no es uno u otro extremo sino un diseño cuidadoso que maximiza el valor forense mientras minimiza el riesgo de privacidad: almacenar el texto completo de input y output cifrado por usuario con TTL definido, mientras los metadatos de seguridad (hashes, scores de clasificadores, IDs de documentos recuperados) se almacenan en un log estructurado de mayor durabilidad y acceso más amplio.

El schema del security event para sistemas de IA es sustancialmente más rico que el de un log HTTP convencional, porque la información relevante para análisis de seguridad incluye componentes específicos de IA que no tienen equivalente en logs de servidor web. Un log de acceso HTTP registra IP, método, path, status code, y latencia. El security event de un sistema de IA debe registrar adicionalmente: el `model_id` y la versión exacta del modelo que procesó la request (porque el mismo endpoint puede servir múltiples versiones del modelo), el hash

del system prompt activo durante la request (no el texto completo, para proteger su confidencialidad pero permitir detectar cambios), los IDs de los documentos recuperados del vectorstore con sus scores de similitud, los nombres y hashes de argumentos de cada herramienta invocada por el agente, y los scores de los clasificadores de seguridad aplicados tanto al input como al output.

El logging de tool calls en sistemas agénticos merece atención especial porque es el registro más crítico para el análisis forense de incidentes en agentes. Cada invocación de herramienta debe registrarse con el nombre de la herramienta, un hash de los argumentos completos (los argumentos completos en un campo separado con cifrado), el resultado (hash del resultado + texto cifrado si es sensible), si la invocación fue autorizada o denegada por el plano de control, y la latencia de la ejecución. Sin este registro, es imposible reconstruir si una secuencia de tool calls fue el resultado de un ataque via tool injection, de un comportamiento inesperado del modelo, o de un uso legítimo del agente.

Aspectos técnicos

- **Schema del security event de IA:** `{event_id, timestamp, user_id, session_id, request_id, model_id, model_version, system_prompt_hash, input_token_count, input_hash, retrieved_document_ids[], retrieved_document_scores[], tool_calls[{name, args_hash, authorized, result_hash, latency_ms}], output_hash, input_safety_score, output_safety_score, latency_ms, error_code}` con texto completo de input/output cifrado por usuario en almacenamiento separado con TTL.
- **Logging de tool calls en agentes:** cada herramienta invocada registrada con nombre, hash de argumentos, estado de autorización (aprobada/denegada), hash del resultado, y latencia; sin este registro es imposible reconstruir un ataque via tool injection.
- **Log levels para seguridad vs. telemetría:** eventos de seguridad (input clasificado como malicioso, rate limit excedido, auth failure, tool call denegada) a nivel ERROR con alerting inmediato; telemetría (request normal, latencias, token counts) a nivel INFO en batch; no mezclar levels para no saturar canales de alerta.
- **Correlación de eventos:** session_id y user_id permiten correlacionar múltiples requests de una sesión o usuario; request_id propagado con W3C Trace Context headers a todos los servicios del pipeline para trazabilidad end-to-end.
- **Logging en streaming:** acumular el output completo antes de registrar el evento final (no un evento por chunk); si se detecta una violación durante el streaming, registrar el evento de violación inmediatamente y truncar el streaming al usuario.

Los logs de seguridad de un sistema de IA son el instrumento de análisis forense más crítico ante un incidente: un log que registra solo "request procesada" sin los IDs de documentos RAG recuperados, los argumentos de tool calls, y los scores de clasificadores de seguridad hace imposible reconstruir si el sistema fue comprometido y cómo.

Módulo 9 – Capítulo 08 – Sección 02

Inmutabilidad de logs: protección contra alteración de registros de auditoría

Los logs de seguridad solo tienen valor como evidencia regulatoria y forense si son inmutables: un log que puede ser modificado o borrado —por el sistema que lo generó, por un administrador con acceso, o por un atacante que comprometió el sistema— no puede usarse para determinar con certeza qué ocurrió durante un incidente, ni para demostrar compliance ante reguladores que preguntan qué controles estaban activos en un momento específico. Esta no es una distinción filosófica: los tribunales y los organismos regulatorios han rechazado logs como evidencia cuando el demandado no podía demostrar que los logs no habían sido alterados después del hecho que se investigaba.

La inmutabilidad de logs es un requisito explícito en múltiples frameworks regulatorios. PCI DSS en su Requirement 10.5 especifica "Secure audit trails so they cannot be altered". HIPAA en su Audit Controls Standard requiere procedimientos de hardware, software y procesos para registrar y examinar accesos a sistemas que contienen PHI, con la implicación de que esos registros deben ser preservados de modificación. El EU AI Act, en sus requisitos de trazabilidad para sistemas de alto riesgo, implica que los registros de decisiones deben poder auditarse con certeza de que no fueron alterados. El GDPR, aunque no especifica técnicamente el mecanismo, exige que los registros de procesamiento de datos personales sean confiables para responder a auditorías regulatorias y a litigios.

La inmutabilidad técnica puede implementarse con diferentes niveles de garantía y diferentes trade-offs de costo y complejidad. El nivel más básico es el almacenamiento write-once: Amazon S3 Object Lock en COMPLIANCE mode previene cualquier modificación o borrado de objetos durante el retention period configurado, incluyendo por usuarios root de la cuenta de AWS —el modo COMPLIANCE es más restrictivo que GOVERNANCE, que permite a administradores con permisos especiales desbloquear objetos. Azure Immutable Blob Storage ofrece capacidades equivalentes con certificación WORM. Para entornos self-hosted o on-premises sin acceso a estos servicios cloud, MinIO (el servidor de object storage open-source

compatible con S3) implementa Object Locking que puede configurarse en modo COMPLIANCE para proporcionar las mismas garantías.

El nivel más robusto añade verificación criptográfica de integridad. Cada entrada de log firmada con HMAC-SHA256 usando una clave de signing almacenada en un KMS separado, donde cada firma incluye el hash de la entrada anterior (creando una cadena de hashes), hace que cualquier modificación —inserción, borrado, o cambio de una entrada— sea detectable porque invalida las firmas de todas las entradas posteriores. Este mecanismo de tamper evidence a nivel de secuencia es más robusto que el write-once storage solo, porque el write-once depende de las garantías del proveedor de storage mientras que la cadena de firmas puede ser verificada matemáticamente por cualquier auditor que tenga acceso a la clave pública de verificación.

Aspectos técnicos

- **Amazon S3 Object Lock en COMPLIANCE mode:** previene modificación o borrado de objetos durante el retention period por cualquier usuario, incluyendo root de AWS; adecuado para logs de auditoría de sistemas de IA bajo GDPR, HIPAA, o PCI DSS; el modo COMPLIANCE es más restrictivo que GOVERNANCE.
- **Azure Immutable Blob Storage:** política time-based retention con lock habilitado que no puede ser reducida una vez aplicada; certificación WORM compatible; adecuado para organizaciones en ecosistema Azure.
- **MinIO Object Locking (self-hosted):** alternativa open-source compatible con S3 API para entornos on-premises o de nube privada; soporta COMPLIANCE mode equivalente al de AWS S3; adecuado para organizaciones sin acceso a cloud público.
- **Firma criptográfica de logs:** cada entrada firmada con HMAC-SHA256 donde la firma incluye el hash de la entrada anterior; cadena de firmas verificable matemáticamente; cualquier modificación invalida las firmas de todas las entradas posteriores.
- **SIEM como copia inmutable secundaria:** transmitir logs a un SIEM (Splunk, Elastic SIEM, Microsoft Sentinel, Wazuh/Graylog para self-hosted) en cuenta o infraestructura independiente como segunda copia inmutable; correlación cross-system y alertas en tiempo casi real.

Nota del Arquitecto: La inmutabilidad de logs debe ser verificada periódicamente, no solo configurada. He auditado sistemas donde S3 Object Lock estaba configurado en modo GOVERNANCE en lugar de COMPLIANCE (modificable por administradores), o donde la retención estaba configurada en 7 días cuando el

requisito regulatorio era 6 años. La verificación automática diaria de que la cadena de firmas es válida, y que el retention period es correcto, convierte la inmutabilidad de un estado declarado en una propiedad verificable del sistema.

La inmutabilidad de logs debe ser una propiedad verificable del sistema —demostrable matemáticamente o mediante auditoría del proveedor de storage— no una propiedad declarada que se asume correcta hasta que alguien pregunta por evidencia.

Módulo 9 – Capítulo 08 – Sección 03

Trazabilidad de decisiones: explicabilidad y atribución en sistemas de IA

La trazabilidad de decisiones en sistemas de IA responde a una pregunta que es simultáneamente un requisito de compliance regulatorio y una necesidad operacional de seguridad: ¿por qué el sistema produjo este output específico dado este input específico? Para los reguladores del EU AI Act (que exige explicabilidad de decisiones automatizadas que afectan a personas en sistemas de alto riesgo), para los usuarios que quieren entender el fundamento de una recomendación, y para los equipos de seguridad que necesitan reconstruir cómo se produjo un incidente, la respuesta a esta pregunta es crítica. La complicación es que la explicabilidad matemática completa de un LLM —por qué exactamente estos pesos con estas atenciones producen exactamente este output dado exactamente este input— es computacionalmente intratable para modelos de producción de millones de parámetros. La trazabilidad práctica, sin embargo, es suficiente para la mayoría de los casos de uso.

La distinción entre explicabilidad y trazabilidad es más que semántica. La explicabilidad intenta responder "por qué el modelo tomó esta decisión" —una pregunta que requiere análisis de las representaciones internas del modelo, análisis de atención, o técnicas de interpretabilidad como SHAP o LIME, que son costosas computacionalmente y no siempre concluyentes para modelos grandes. La trazabilidad registra "qué inputs, herramientas, documentos y contexto estaban presentes cuando el sistema produjo este output" —una pregunta que puede responderse completamente con los logs correctamente diseñados, sin análisis de internals del modelo. Para la mayoría de los requisitos regulatorios y forenses, la trazabilidad es suficiente: el auditor o el investigador de seguridad puede ver exactamente qué información tenía el modelo disponible cuando generó el output en cuestión.

El chain of thought logging es la mejor aproximación disponible a la explicabilidad del proceso de decisión en sistemas que usan razonamiento explícito. En modelos que soportan extended thinking —Claude 3.7 Sonnet, GPT-o1, o modelos con chain-of-thought prompting explícito— el razonamiento intermedio del modelo antes de su respuesta final debe registrarse como parte del log del request. Este razonamiento no es la explicación matemática de la

decisión, pero proporciona el proceso de inferencia del modelo: qué consideró, qué descartó, qué incertidumbre tenía. En análisis forense, la diferencia entre un output que fue el resultado de razonamiento coherente y uno que fue el resultado de un jailbreak que interrumpió el razonamiento normal puede ser visible en el chain of thought.

La atribución de documentos RAG —registrar qué documentos del corpus fueron recuperados y qué fragmentos específicos el modelo usó en su respuesta— es un mecanismo de explicabilidad aplicado específicamente a sistemas RAG. LlamaIndex, LangChain, y otros frameworks soportan citation generation: la respuesta del modelo incluye referencias explícitas a los chunks de documentos que fundamentaron cada afirmación. Este mecanismo es simultáneamente explicabilidad para el usuario (sabe de dónde viene la información), trazabilidad para el auditor (puede verificar que la información en la respuesta corresponde a los documentos del corpus), y control de seguridad (puede detectar si el modelo "alucinó" información sin fuente verificable).

Aspectos técnicos

- **Chain of thought logging:** registrar el razonamiento intermedio del modelo (extended thinking, CoT explícito) como parte del log del request; crítico para análisis forense (¿el modelo razonó correctamente o fue manipulado?) y para compliance de explicabilidad en sistemas de alto riesgo del EU AI Act.
- **Citation generation para atribución de documentos RAG:** cada afirmación factual atribuida a un chunk de documento específico con su ID y score de similitud; LlamaIndex y LangChain soportan esto nativamente; simultáneamente explicabilidad para el usuario y trazabilidad para el auditor.
- **Versioning del system prompt:** registrar con qué versión exacta del system prompt se procesó cada request; un cambio en el system prompt puede cambiar significativamente el comportamiento del modelo, y sin versioning es imposible distinguir un ataque de un cambio no autorizado del prompt.
- **Model card y version tracking por request:** model_id + versión registrados en cada request; en sistemas con múltiples modelos (clasificador de input + LLM generativo + clasificador de output), cada componente versionado y su versión registrada en el log del request completo.
- **Decision audit trail para sistemas de alto impacto:** en sistemas donde la IA toma o influye en decisiones que afectan a personas (préstamos, diagnósticos, contratación), el audit trail debe incluir output del modelo, documentos de contexto, confidence scores,

cualquier override humano, y el outcome final — suficiente para revisión regulatoria post-hoc del EU AI Act.

La trazabilidad de decisiones no es sinónimo de explicabilidad matemática del modelo sino el registro completo de las condiciones bajo las que se produjo cada output: versión del modelo, versión del system prompt, documentos recuperados, herramientas invocadas, y scores de clasificadores — suficiente para análisis forense de incidentes y para compliance regulatorio de sistemas de alto riesgo.

Módulo 9 – Capítulo 08 – Sección 04

SIEM integration: incorporar eventos de IA a los sistemas de monitoreo de seguridad

Los logs de seguridad de un sistema de IA, analizados en aislamiento, revelan patrones de seguridad que son visibles solo en el contexto de ese sistema: un usuario que hace múltiples requests con características de jailbreak, un patrón de queries con alta diversidad semántica que sugiere model extraction, o una secuencia de tool calls que podría indicar un agent injection exitoso. Pero muchos ataques a sistemas de IA no son ataques aislados: son parte de campañas más amplias que incluyen actividad maliciosa en otros sistemas de la organización. Un intento de model extraction puede correlacionarse con un acceso sospechoso a la cuenta de AWS desde una IP nueva. Un jailbreak exitoso puede correlacionarse con un acceso desde una IP de una red Tor. La integración con el SIEM corporativo —Splunk, Microsoft Sentinel, Elastic SIEM, IBM QRadar, o Wazuh/Graylog para entornos self-hosted— es el mecanismo que permite estas correlaciones cross-system.

La integración típica usa formatos estándar de intercambio de eventos de seguridad. CEF (Common Event Format) es el formato más adoptado en SIEMs maduros y proporciona una estructura de campos predefinida con extensiones para campos personalizados. OCSF (Open Cybersecurity Schema Framework), adoptado por AWS, Splunk, IBM, y otros en 2022, es el estándar emergente con mayor capacidad de extensión para casos de uso específicos como IA. Para sistemas de IA, los campos más críticos del evento SIEM son: el tipo de evento (PromptInjectionAttempt, JailbreakAttempt, RateLimitExceeded, SafetyFilterTriggered, ModelExtractionSuspected, ToolCallDenied), la severidad (0-10), el user_id, la IP del cliente, el model_id, y los campos de extensión específicos de IA (input_safety_score, retrieved_document_ids, tool_calls).

Las reglas de correlación en el SIEM son donde el valor operacional de la integración se materializa. La correlación requiere pensar en términos de escenarios de ataque completos, no en términos de eventos individuales: un rate limit excedido más una IP nunca vista previamente es una señal de possible model extraction; múltiples safety filter triggers del mismo user_id en 24 horas es una señal de jailbreak sistemático; una prompt injection

detectada más una tool call a un endpoint externo no en la allowlist es una señal de posible exfiltración via agente. Cada una de estas correlaciones debe convertirse en una regla en el SIEM con su umbral de activación y su acción (alerta, bloqueo temporal, escalation a revisión manual).

Los entornos self-hosted o on-premises tienen alternativas completas a los SIEMs cloud: Wazuh (basado en OSSEC) es una plataforma de SIEM/XDR open-source con reglas de correlación configurables y capacidad de ingestión de logs en formato CEF; Graylog es una alternativa open-source orientada a log management con capacidades de alerting. Ambas pueden instalarse on-premises y configurarse para recibir los eventos de seguridad del sistema de IA, aplicar reglas de correlación, y enviar alertas al equipo de seguridad. La diferencia con SIEMs cloud es la responsabilidad operacional: el equipo gestiona la infraestructura del SIEM, las actualizaciones, y el escalado.

Aspectos técnicos

- **Formato de eventos para SIEM:** CEF (Common Event Format) para compatibilidad con SIEMs maduros (Splunk, QRadar), OCSF (Open Cybersecurity Schema Framework) como estándar emergente adoptado por AWS, Splunk, y otros; campos de extensión específicos de IA (input_safety_score, retrieved_document_ids, tool_calls).
- **Reglas de correlación para sistemas de IA:** rate limit excedido + IP nueva = posible model extraction; safety filter triggers repetidos del mismo user_id = jailbreak sistemático; prompt injection detectada + tool call a endpoint externo no autorizado = posible exfiltración via agente.
- **Pipeline de integración:** logs del sistema de IA (JSON en CloudWatch/Azure Monitor/stdout) → transformación con Fluentd/Logstash a formato CEF/OCSF → ingestión al SIEM con latencia <60 segundos para alertas en tiempo casi real.
- **Alternativas on-premises:** Wazuh (OSSEC-based, open-source) con capacidades SIEM/XDR + reglas de correlación configurables; Graylog para log management + alerting; ambos reciben eventos en CEF y soportan correlación cross-source.
- **Dashboards de seguridad específicos para IA:** volumen de requests por tiempo y tenant, distribución de safety filter triggers por categoría, top usuarios por intentos de jailbreak, latencia P99 de clasificadores de seguridad, y rate limits excedidos por IP y tenant.

Nota del Arquitecto: El valor principal de la integración SIEM no es el alerting de eventos individuales —eso puede hacerse con CloudWatch Alarms o Azure Monitor

Rules sin un SIEM completo— sino la correlación cross-system. El mismo atacante que intenta model extraction en el sistema de IA probablemente también probó acceder a otros sistemas de la organización. Sin integración SIEM, esos eventos son puntos de datos aislados en sistemas diferentes que nadie correlaciona. Con integración SIEM, son parte de un patrón de ataque identificable.

La integración con SIEM convierte los eventos de seguridad del sistema de IA en parte del programa de seguridad corporativo, permitiendo que las amenazas específicas de IA sean detectadas y respondidas con las mismas herramientas y procesos que otras amenazas, eliminando el punto ciego que un sistema de IA sin integración SIEM crea en el landscape de seguridad de la organización.

Módulo 9 – Capítulo 08 – Sección 05

Incident response para sistemas de IA: detección, contención y análisis forense

El incident response (IR) para sistemas de IA comparte el framework general del NIST SP 800-61 —Preparación → Detección y Análisis → Contención, Erradicación y Recuperación → Post-incident— pero sus procedimientos específicos son sustancialmente diferentes de los del IR de aplicaciones web o sistemas de red. Un "incidente" en un sistema de IA tiene características únicas que lo hacen más difícil de detectar y de contener: puede ser difícil de distinguir de comportamiento normal (un modelo comprometido por data poisoning se comporta normalmente el 99.9% del tiempo), puede tener impacto difuso sobre múltiples usuarios sin que ningún usuario individual lo reporte (un documento malicioso en el vectorstore afecta a todos los usuarios que hacen queries relacionadas), y la "contención" puede requerir decisiones de negocio complejas que involucran la disponibilidad del servicio (¿desactivar el sistema de IA en producción mientras se investiga?).

La fase de preparación pre-incidente es la más crítica y la más frecuentemente subestimada. Cuando un incidente ocurre, el equipo debe poder actuar en minutos, no en horas que se pierden buscando documentación o pidiendo accesos. Los runbooks específicos para los incidentes más probables deben estar documentados, revisados, y probados (mediante simulacros anuales) antes de que el sistema entre en producción. Los runbooks mínimos necesarios para un sistema de IA son: jailbreak activo en producción (usuarios están recibiendo outputs problemáticos del modelo), RAG poisoning detectado (hay evidencia de un documento malicioso en el vectorstore), data leakage (el modelo está revelando datos que no debería), prompt leaking activo (el system prompt está siendo extraído en las respuestas), y model extraction sospechada (patrones de queries sugieren que alguien está replicando el comportamiento del modelo sistemáticamente).

Los indicadores de compromiso (IoC) específicos de sistemas de IA son el mecanismo que permite la detección proactiva antes de que el impacto sea visible para los usuarios. Un spike en la tasa de safety filter triggers que supera el doble de la línea de base histórica debe trigger alerta. Un aumento en requests con características de model extraction —alta diversidad

semántica, cobertura sistemática de categorías de temas— debe trigger alerta. Outputs que contienen el texto del system prompt deben trigger alerta inmediata. Tool calls a endpoints externos no en la allowlist deben trigger alerta y bloqueo. Queries con patrones de membership inference (textos muy similares a secuencias conocidas del training data del modelo) deben trigger revisión manual. Estos umbrales deben estar configurados en el SIEM con reglas de correlación activas, no en un documento de seguridad que nadie consulta.

La contención de un sistema de IA comprometido tiene opciones de escalada que permiten calibrar la respuesta al nivel de confianza en el diagnóstico y al impacto del servicio. El nivel más conservador es activar el modo restrictivo del output filter —subir el umbral de bloqueo del clasificador para que más outputs sean bloqueados. El siguiente nivel es deshabilitar herramientas específicas del agente que se cree que fueron el vector de ataque. El siguiente es ejecutar rollback al checkpoint del modelo anterior si hay evidencia de data poisoning. El siguiente es failover a un sistema fallback más simple (un sistema de respuestas predefinidas o un modelo más pequeño y conservador). El nivel más extremo es la desactivación completa con mensaje de mantenimiento mientras se investiga. Cada opción tiene un trade-off explícito entre seguridad y disponibilidad que debe decidirse en el contexto del incidente, no improvisarse en el momento.

Aspectos técnicos

- **Runbooks específicos para incidentes de IA:** jailbreak activo, RAG poisoning detectado, data leakage, prompt leaking, model extraction sospechada; cada runbook con pasos de detección, contención, erradicación, recuperación, y notificación a reguladores si aplica.
- **IoCs de sistemas de IA:** spike en safety filter trigger rate (>2x baseline), patterns de model extraction (alta diversidad semántica), outputs que contienen el system prompt, tool calls a endpoints no autorizados, queries con patterns de membership inference.
- **Opciones de contención escaladas:** modo restrictivo del output filter → deshabilitar herramientas específicas → rollback al modelo anterior → failover a sistema fallback simple → desactivación completa; cada nivel con su trade-off explícito de seguridad vs. disponibilidad.
- **Análisis forense post-incidente:** reconstrucción completa de la secuencia del ataque usando el audit trail (model_id, user_id, retrieved_document_ids, tool_calls, safety_scores); identificación del vector de entrada, técnica utilizada, controles que fallaron; mejoras concretas documentadas para el sistema.

- **Post-incident activities:** notificación a DPA bajo GDPR Art. 33 (72h si hay data breach de usuarios europeos), notificación a usuarios afectados si aplica Art. 34, actualización de runbooks con lecciones aprendidas, y actualización del threat model con las nuevas técnicas de ataque observadas.

El incident response para sistemas de IA debe prepararse antes de que ocurra el incidente: los runbooks, los umbrales de alerta en el SIEM, los procedimientos de contención escalonados, y los contactos de escalation deben estar documentados, probados, y actualizados antes de que el sistema entre en producción.

Módulo 9 – Capítulo 08 – Sección 06

Cierre: sin trazabilidad no hay auditoría posible ni aprendizaje de incidentes

La trazabilidad y la auditoría en sistemas de IA no son capacidades que se añaden por compliance regulatorio externo sino herramientas de ingeniería que mejoran activamente la capacidad del equipo para entender, operar, y mejorar el sistema. Un sistema de IA sin trazabilidad se opera con los ojos cerrados: no se sabe si está siendo atacado ahora mismo, no se puede saber con certeza qué ocurrió cuando algo sale mal, y no se puede aprender de los incidentes para mejorar los controles porque no hay evidencia del incidente. La inversión en un sistema de logging e auditoría robusto —schema bien diseñado, almacenamiento inmutable, integración con SIEM, playbooks de IR— no es un gasto de compliance; es una inversión en capacidad operacional que tiene ROI medible en tiempo de detección de incidentes reducido, tiempo de respuesta reducido, y menor severidad de incidentes futuros porque las lecciones de incidentes pasados se incorporaron al sistema.

El ciclo de mejora continua que habilita la trazabilidad funciona de manera específica: el log de seguridad registra un intento de jailbreak detectado por el input filter; el análisis del log revela que la técnica era una variante de Crescendo attack que el filter identificó con un score bajo (casi falso negativo); el equipo calibra el umbral del classificador basado en esta observación; el red teaming periódico verifica que la calibración fue correcta; y el threat model se actualiza para incluir esta variante de Crescendo attack como amenaza documentada. Sin el log detallado con el safety score y la técnica detectada, esta cadena de mejora no puede iniciarse. La trazabilidad es el prerrequisito de la mejora continua del posture de seguridad.

Los sistemas de IA sin trazabilidad adecuada también presentan un problema específico ante auditorías regulatorias: no pueden demostrar que los controles estuvieron activos en el momento relevante. Un auditor de GDPR que pregunta "¿qué datos del usuario X fueron procesados por su sistema de IA entre las fechas A y B?" necesita una respuesta auditable con evidencia. Sin logs con los campos correctos, la única respuesta posible es "no tenemos

registros" —que puede ser interpretada como incumplimiento en lugar de como ausencia de incidente.

Conceptos clave del capítulo

- **Schema de security events:** user_id + session_id + model_id + model_version + system_prompt_hash + retrieved_document_ids[] + tool_calls[] + input_safety_score + output_safety_score — mínimo vital para análisis forense y compliance GDPR/HIPAA; texto completo cifrado por usuario con TTL.
- **Inmutabilidad de logs:** S3 Object Lock COMPLIANCE mode (AWS), Azure Immutable Blob Storage, MinIO Object Locking para self-hosted; firma criptográfica HMAC-SHA256 con cadena de hashes para tamper evidence verificable matemáticamente.
- **Trazabilidad de decisiones:** CoT logging para razonamiento intermedio del modelo, citation generation para atribución de fuentes RAG, versioning del system prompt y model card tracking por request.
- **SIEM integration:** formato CEF/OCSF, reglas de correlación cross-system (rate limit + IP nueva = extracción; safety triggers repetidos = jailbreak sistemático), dashboards específicos para IA; Wazuh/Graylog para entornos self-hosted.
- **Incident response:** runbooks específicos por tipo de incidente (jailbreak, RAG poisoning, data leakage), IoCs de IA como umbrales de alerta en el SIEM, opciones de contención escalonadas por impacto, análisis forense con mejoras documentadas.

"Logging is not just for debugging — it is the historical record of your system's behavior, and without it you are flying blind into the past." — Kelsey Hightower, Staff Developer Advocate en Google Cloud y uno de los ingenieros más influyentes en cloud-native architecture, sobre el rol fundamental del logging en sistemas de producción: no es un accesorio para cuando algo falla, sino el registro continuo que permite saber qué está pasando y qué pasó.